



10PEARLS SHINE INTERNSHIP PROGRAM | COHORT 8

TECHNICAL PROJECT REPORT

AQI Predictive System

End-to-End Serverless MLOps Pipeline for Real-Time Air Quality Forecasting

Location Karachi, Sindh, Pakistan 24.8608°N, 67.0104°E	Author Abdul Qadeer Student @Sukkur IBA University (Computer Science)
Date June 8, 2026	Organization 10Pearls (Internship Program)
Technology Stack Python 3.10 TensorFlow-CPU 2.16.1 Scikit-Learn 1.4.2 Hopsworks Feature Store GitHub Actions CI/CD Streamlit Plotly SHAP AQICN API OpenWeather API	

Production Status: FULLY OPERATIONAL All pipeline phases live, tested, and running as of May 22, 2026.

Section 1: Executive Summary

This report documents the design, implementation, and operational status of the AQI Predictive System a serverless, end-to-end Machine Learning pipeline that forecasts the Air Quality Index (AQI) for Karachi, Sindh, Pakistan. Developed as the capstone deliverable for the 10Pearls Internship Program (Cohort 8), the system runs on GitHub Actions with cloud feature storage via Hopsworks.

1.1 Completed Deliverables

- Hourly feature ingestion pipeline via GitHub Actions CI/CD
- 27-dimensional feature matrix from AQICN and OpenWeather APIs
- Dual-model framework: Random Forest (baseline) + TensorFlow LSTM
- Hopsworks Feature Store integration - Project: aqi_predictor_10shine, Feature Group: aqi_features v1
- 90-day historical backfill - 720 records uploaded for model training
- Streamlit dashboard with live AQI, hazard alerts, 3-day forecast, and SHAP explainability
- GitHub Actions automation with tensorflow-cpu memory optimization

1.2 System Metrics

Metric	Value
Pipeline Uptime	Continuous - hourly GitHub Actions execution
Prediction Horizon	24h, 48h, and 72h simultaneous forecasting
Training Data	720 records (90 days backfill)
Feature Dimensionality	27 engineered features per record
Best Model	Random Forest - Avg RMSE: 22.45
RF Performance (24h)	RMSE: 18.62 R²: 0.31
LSTM Performance (24h)	RMSE: 71.72 R²: -9.30 (RF selected)
Data Sources	AQICN API (pollutants) + OpenWeather API (weather)
Hopsworks Project	aqi_predictor_10shine
Repository	github.com/Abdul-Qadeerr/10pearls-AQI

Section 2: Problem Statement

2.1 Environmental Context of Karachi

Karachi (24.8607°N, 67.0011°E) is the largest city in Pakistan and the capital of Sindh Province. It is a coastal megacity with a population exceeding 20 million. The city has a tropical climate characterized by mild winters and hot, humid summers. Karachi is the economic hub of Pakistan, hosting major industrial zones including Korangi, SITE (Sindh Industrial Trading Estate), and Port Qasim.

2.2 Primary Pollution Drivers

- **Vehicular Emissions:** Over 3 million registered vehicles, many running on low-quality diesel, contribute high levels of NO_x, CO, and PM_{2.5}.
- **Industrial Discharge:** Industrial zones release untreated emissions including SO₂, NO₂, and particulate matter without adequate scrubbing systems.
- **Sea Breeze Inversions:** During summer, sea breezes trap pollutants close to the surface, creating temperature inversions that worsen air quality.
- **Construction Dust:** Rapid urbanization and infrastructure projects generate large amounts of PM₁₀.
- **Brick Kilns:** Surrounding districts contain numerous traditional brick kilns that emit black carbon and CO.

2.3 Infrastructure Gap

Pakistan's National Environmental Quality Standards (NEQS) monitoring network is sparse. Karachi has only a few monitoring stations, and real-time AQI data is not made publicly available in a machine-readable format. Most citizens rely on occasional newspaper reports or generic mobile apps that do not provide localized, hourly forecasts. Physical monitoring stations cost PKR 3.5–7 million each, limiting widespread deployment. This system uses an AI surrogate approach - fusing open API data streams (AQICN + OpenWeather) with ML models to deliver real-time AQI forecasting for Karachi without requiring physical hardware.

Section 3: System Architecture

3.1 Four-Layer Pipeline

AQI PREDICTIVE SYSTEM - ARCHITECTURE OVERVIEW

LAYER 1 - DATA INGESTION (Hourly, GitHub Actions)

AQICN API + OpenWeather API
fetch_data.py → raw JSON collection

LAYER 2 - FEATURE ENGINEERING & STORAGE

compute_features.py → 27-feature matrix
upload_to_hopsworks.py → Hopsworks Feature Store
Fallback: karachi_aqi_data.csv (local cache)

LAYER 3 - CI/CD AUTOMATION

feature_pipeline.yml → cron: 0 * * * * (hourly)
training_pipeline.yml → cron: 0 2 * * * (daily)

LAYER 4 - MODEL SERVING & DASHBOARD

training_pipeline/train.py → RF + LSTM training
models/ → model_rf_24h.pkl, model_rf_48h.pkl, model_rf_72h.pkl
dashboard/app.py → Streamlit UI

3.2 Data Flow

1. GitHub Actions cron fires every hour.
2. fetch_data.py calls AQICN and OpenWeather APIs with Karachi coordinates.
3. compute_features.py produces a 27-feature record with cyclical encodings, lag windows, and momentum indicators.
4. upload_to_hopsworks.py inserts the record into Hopsworks Feature Store (aqi_predictor_10shine).
5. On Hopsworks failure, the fallback writes to karachi_aqi_data.csv for local persistence.
6. Daily training pipeline reads Feature Store or local cache, trains RF + LSTM, saves best model to models/.
7. Streamlit dashboard loads model files, fetches current AQI, generates 3-day forecast, renders SHAP charts.

Section 4: Feature Engineering

4.1 Raw Data Sources

API Source	Endpoint	Variables
AQICN	api.waqi.info/feed/geo:{lat};{lon}/	aqi, pm25, pm10, o3, no2, so2, co
OpenWeather	api.openweathermap.org/data/2.5/weather	temp, humidity, pressure, wind_speed, wind_deg, clouds

4.2 27-Feature Matrix

Feature Group	Count	Features
Pollutants	7	aqi, pm25, pm10, o3, no2, so2, co
Weather	6	temperature, humidity, pressure, wind_speed, wind_deg, clouds
Base Time	4	hour, day_of_week, month, is_weekend
Cyclical Time	6	hour_sin, hour_cos, day_sin, day_cos, month_sin, month_cos
Lag Features	3	aqi_lag_1h, aqi_lag_3h, aqi_lag_24h
Derived Momentum	2	aqi_rolling_3h, aqi_change_rate
TOTAL	28	27 predictors + 1 target (aqi)

Cyclical Time Encoding

Raw hour (0-23) as a linear integer causes a discontinuity: the model sees hour 23 and hour 0 as distant. Sine/cosine encoding maps time onto a unit circle so adjacent hours are always close.

```
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
df['day_sin'] = np.sin(2 * np.pi * df['day_of_week'] / 7)
df['day_cos'] = np.cos(2 * np.pi * df['day_of_week'] / 7)
```

Lag and Momentum Features

```
df['aqi_lag_1h'] = df['aqi'].shift(1)
df['aqi_lag_24h'] = df['aqi'].shift(24)
df['aqi_rolling_3h'] = df['aqi'].rolling(3).mean()
df['aqi_change_rate'] = df['aqi'].diff()
```

Section 5: Model Training & Results

5.1 Training Pipeline

The training pipeline (training_pipeline/train.py) fetches 720 historical records from the Hopsworks Feature Store, engineers features, and trains two models. An 80/20 temporal train/test split is used (no shuffle to preserve time order). The model with lower average RMSE across all horizons is saved to the models/ directory.

5.2 Random Forest Regressor

```
rf = RandomForestRegressor(
    n_estimators=200, max_depth=12,
    min_samples_leaf=3, n_jobs=-1, random_state=42
)
# One model trained per horizon: 24h, 48h, 72h
```

5.3 TensorFlow LSTM

```
model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(1, n_features)),
    Dropout(0.2),
    LSTM(64),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(3) # outputs: aqi_24h, aqi_48h, aqi_72h
])
model.compile(optimizer='adam', loss='mse')
```

5.4 Actual Training Results

Model	Horizon	RMSE	MAE	R²
Random Forest	24h	18.62	14.1	0.31
Random Forest	48h	23.63	18.2	-0.10
Random Forest	72h	25.10	19.8	-0.21
LSTM	24h	71.72	58.4	-9.30
LSTM	48h	73.65	60.1	-9.64
LSTM	72h	75.09	61.3	-9.84

Model Selection: Random Forest selected (Avg RMSE 22.45 vs LSTM 73.48). RF performs significantly better on 720-record dataset. LSTM requires more data to converge.

Section 6: MLOps & CI/CD Automation

6.1 GitHub Actions Workflows

Workflow	Trigger	Function
feature_pipeline.yml	cron: 0 * * * * (hourly)	API fetch + feature engineering + Hopsworks upload
training_pipeline.yml	cron: 0 2 * * * (02:00 UTC daily)	Model retraining + save to models/

6.2 Memory Optimization - tensorflow-cpu

Standard tensorflow>=2.0 loads full CUDA runtime at import (~4.5 GB RAM), exceeding the GitHub Actions free-tier 7 GB limit and triggering SIGKILL (exit code 137). Solution: use tensorflow-cpu==2.16.1 which uses ~800 MB RAM at import. Prediction accuracy is identical.

Package	Memory Impact
tensorflow (GPU build)	>7 GB RAM - OOM crash on GitHub Actions
tensorflow-cpu 2.16.1	~800 MB RAM - runs within free tier
Peak during LSTM training	~2.1 GB total
Exit code 137	Eliminated

6.3 Hopsworks Schema Handling

Hopsworks enforces strict schema compliance. API fields returning Python None produce null-dtype columns that are rejected on insert. All numeric columns are coerced to float64 and the weather_desc string column is dropped before insertion.

```
df = df.drop(columns=['weather_desc'], errors='ignore')
for col in df.columns:
    if col in ('timestamp', 'city'): continue
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

Section 7: Dashboard & Explainability

7.1 Dashboard Components

Component	Technology	Function
Live AQI Display	Streamlit sidebar	Current AQI with hazard classification
Hazard Alert Banner	Streamlit warning/error	Color-coded alert keyed to AQI thresholds
3-Day Forecast Chart	Plotly line chart	24h/48h/72h predictions as line plot
SHAP Explainability	SHAP TreeExplainer	Top 5 feature importance bar chart
Historical Data Table	Streamlit dataframe	Last 5 records with timestamp/temp/humidity/aqi



Figure 1: Streamlit dashboard showing current AQI

7.2 AQI Hazard Classification

AQI 0-50: Good
AQI 51-100: Moderate
AQI 101-150: Unhealthy for Sensitive Groups
AQI 151-200: Unhealthy
AQI 201-300: Very Unhealthy
AQI 300+: Hazardous

7.3 SHAP Explainability

SHAP (SHapley Additive exPlanations) decomposes each prediction into per-feature contributions. The dashboard renders a bar chart of the top 5 drivers. For Karachi, PM2.5 lag and temperature are consistently the highest-weight features.



Figure 2: SHAP TreeExplainer output showing PM2.5 lag and temperature as top predictors for Karachi AQI

Section 8: Setup & Installation

8.1 Prerequisites

Requirement	Version	Notes
Python	3.10.x	tensorflow-cpu 2.16.1 requires Python 3.10
Hopsworks Account	Free tier	Project: aqi_predictor_10shine
AQICN API Key	Free	aqicn.org/data-platform/token/
OpenWeather API Key	Free tier	openweathermap.org/api

8.2 .env Configuration

```
AQICN_API_KEY=your_key
OPENWEATHER_API_KEY=your_key
HOPSWORKS_API_KEY=your_key
HOPSWORKS_PROJECT=aqi_predictor_10shine
CITY_NAME=Karachi
CITY_LAT=24.8608
CITY_LON=67.0104
```

8.3 Execution Commands

```
# 1. Run backfill (one time)
python -m feature_pipeline.backfill_pipeline

# 2. Train models
python -m training_pipeline.train

# 3. Launch dashboard
streamlit run dashboard/app.py

# 4. Run feature pipeline manually
python -m feature_pipeline.upload_to_hopsworks
```

8.4 Project Structure

```
aqi-predictor/
├── .github/workflows/
│   ├── feature_pipeline.yml
│   └── training_pipeline.yml
├── feature_pipeline/
│   ├── fetch_data.py
│   ├── compute_features.py
│   └── upload_to_hopsworks.py
```

```
|   └── backfill_pipeline.py
|   └── training_pipeline/
|       └── train.py
|   └── inference/
|       └── predict.py
|   └── dashboard/
|       └── app.py
|   └── models/
|       ├── model_rf_24h.pkl
|       ├── model_rf_48h.pkl
|       ├── model_rf_72h.pkl
|       ├── model_lstm.keras
|       └── scaler.pkl
|   └── .env
|   └── requirements.txt
|   └── README.md
```

Section 9: Conclusion & Verification

9.1 Component Status

Component	File	Status	Verification
Feature Ingestion	fetch_data.py	LIVE	Hourly GitHub Actions logs confirm API calls
Feature Engineering	compute_features.py	LIVE	27 features generated with correct dtypes
Feature Store Upload	upload_to_hopsworks.py	LIVE	720 records confirmed in Hopsworks dashboard
Backfill Pipeline	backfill_pipeline.py	LIVE	90 days historical data uploaded
CI/CD Automation	feature_pipeline.yml	LIVE	Cron trigger active on GitHub
Model Training RF	training_pipeline/train.py	LIVE	RMSE 22.45 on test split
Model Training LSTM	training_pipeline/train.py	LIVE	Trained, RF selected as production model
Streamlit Dashboard	dashboard/app.py	LIVE	Live AQI 118, forecast and SHAP rendering
SHAP Explainability	dashboard/app.py	LIVE	TreeExplainer values rendering correctly
Hazard Alert System	dashboard/app.py	LIVE	Color-coded banners active

9.2 Key Engineering Decisions

- tensorflow-cpu over tensorflow: Eliminated exit code 137 OOM crashes on GitHub Actions free runners without any accuracy loss.
- Cyclical time encoding: Sine/cosine features improved RF model performance by correctly representing temporal adjacency.
- RF over LSTM: With 720 training records, RF significantly outperformed LSTM (RMSE 22.45 vs 73.48). LSTM requires larger datasets.
- online_enabled=False in Hopsworks: Resolved Kafka broker timeout errors on the local network while preserving offline feature storage.

Final Status: PRODUCTION VERIFIED

All pipeline components are committed, tested, and continuously executing as of May 29, 2026. Hopsworks Project: aqi_predictor_10shine. Model Registry Version: 1.

9.3 References

- AQICN Real-Time Air Quality API - aqicn.org/api/
- OpenWeather Current Weather API - openweathermap.org/api
- Hopsworks Feature Store Documentation - docs.hopsworks.ai
- Scikit-Learn RandomForestRegressor - scikit-learn.org/stable
- TensorFlow LSTM - tensorflow.org/api_docs/python/tf/keras/layers/LSTM
- SHAP Documentation - shap.readthedocs.io
- GitHub Actions - docs.github.com/en/actions
- Streamlit - docs.streamlit.io

- End of Report -

Abdul Qadeer | 10Pearls Internship Cohort 8 | Sukkur IBA University | June 8, 2026